



Nagar Sheba

City Complaint & Service Request Platform

REST API Documentation

Base URL: /api/v1 · 52 Endpoints · Version 1.0

B7A6 Backend Project Assignment — City Complaint & Service Platform

Generated September 2026

Table of Contents

1. Overview

2. Authentication & Authorization

3. Standard Response Format

4. Roles & Permissions

5. API Reference

1. Authentication
2. Departments
3. Categories
4. Service Requests
5. Payments
6. Feedback
7. Admin
8. Notifications

6. Data Model Summary

7. Error Codes Reference

1. Overview

Nagar Sheba is a backend-only RESTful API that lets citizens report civic issues (potholes, water leakage, garbage collection, licensing requests, etc.), routes them to the right city department, tracks them through a full status lifecycle, and handles paid service fees through real payment gateways (SSLCommerz and bKash).

Tech Stack

- Runtime & Framework: Node.js, TypeScript, Express.js
- Database & ORM: PostgreSQL, Prisma ORM (@prisma/adapters-pg)
- Validation: Zod
- Authentication: JWT (access + refresh, Bearer token) + Google OAuth (GCP)
- Caching / Temp State: Redis (OTP storage, rate-limit backing)
- File Storage: Multer (memory storage) + Cloudinary
- Email: Nodemailer (Gmail SMTP) + EJS templates
- Payments: SSLCommerz, bKash Tokenized Checkout
- Security: Helmet, express-rate-limit (Redis-backed), cookie-parser, CORS
- Deployment: Vercel Serverless Functions / Render

Core Workflow

- Citizen creates a Service Request under a Category.
- PAID category → status starts at PENDING_PAYMENT → citizen completes checkout → SUBMITTED.
- FREE category → status starts directly at SUBMITTED.
- Admin reassigns / Staff picks up the request → ASSIGNED (SLA due date computed from the category's slaHours).
- Staff moves the request → IN_PROGRESS → RESOLVED (resolution note required).
- Citizen may reopen a RESOLVED request within 3 days → back to ASSIGNED, or it auto-closes to CLOSED after 3 days via a background job.
- Citizen may CANCEL while PENDING_PAYMENT / SUBMITTED / ASSIGNED — a completed payment is automatically refunded.
- Admin may override the status at any non-terminal point.

Background Jobs

`runRequestLifecycleJob` runs at startup and then every 15 minutes:

- Flags any non-terminal request whose slaDueAt has passed as isOverdue.
- Auto-closes a RESOLVED request into CLOSED once the 3-day citizen response window elapses.

2. Authentication & Authorization

All protected routes require a Bearer access token: Authorization: Bearer <accessToken> — the middleware also falls back to an httpOnly accessToken cookie (set automatically on login/registration for browser clients).

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

On every request, the role-based auth(...) middleware re-verifies the JWT, then re-fetches the user from the database and rejects the request if the account has since been soft-deleted or blocked — a still-valid token cannot be used against a deactivated account.

Token Lifetimes

- Access Token — short-lived (configurable via JWT_ACCESS_EXPIRES_IN, default 1 day)
- Refresh Token — longer-lived (configurable via JWT_REFRESH_EXPIRES_IN, default 7 days), rotated on every use via POST /auth/refresh-token

3. Standard Response Format

Every endpoint returns one consistent JSON envelope.

Success

```
{
  "success": true,
  "statusCode": 200,
  "message": "Service request fetched successfully",
  "data": {},
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 42,
    "totalPages": 5
  }
}
```

`meta` is only present on paginated list endpoints.

Error

```
{
  "success": false,
  "statusCode": 400,
  "message": "Validation failed",
  "errors": [
    {
      "path": "email",
      "message": "Please provide a valid email address."
    }
  ]
}
```

4. Roles & Permissions

Role	Description	Key Permissions
CITIZEN	Reports issues and pays applicable fees	Create/cancel/reopen own requests, upload evidence, pay fees, leave feedback, manage own profile
STAFF	Department field worker assigned to requests	View requests scoped to their own department; move an assigned request ASSIGNED → IN_PROGRESS → RESOLVED; attach resolution proof
ADMIN	Platform operator	Manage departments & categories; provision staff/admin accounts; reassign / override any request status; block/unblock users; view audit logs & dashboard stats

5. API Reference

The API exposes 52 endpoints across 8 modules, versioned under `/api/v1`. Each module below lists a quick-reference table followed by full detail (request body, response shape, and business rules) for every endpoint.

1. Authentication

Base path: `/api/v1/auth`

Handles citizen self-registration (OTP email verification), Google OAuth sign-in/sign-up, credential login, password recovery, token refresh, and profile management. Successful login/registration sets `httpOnly` `accessToken` and `refreshToken` cookies in addition to returning the tokens in the JSON body.

Method	Endpoint	Access	Description
POST	<code>/api/v1/auth/register</code>	Public	Start citizen registration (sends OTP email)
POST	<code>/api/v1/auth/verify-email</code>	Public	Verify OTP, create account, return tokens
POST	<code>/api/v1/auth/google-login</code>	Public	Google ID token sign-in / sign-up (Citizen only)
POST	<code>/api/v1/auth/login</code>	Public	Email/password login (any role)
POST	<code>/api/v1/auth/forgot-password</code>	Public	Send a password-reset OTP
POST	<code>/api/v1/auth/reset-password</code>	Public	Verify OTP and set a new password
GET	<code>/api/v1/auth/me</code>	Authenticated (any role)	Get the current user's profile
PATCH	<code>/api/v1/auth/me</code>	Authenticated (any role)	Update the current user's profile
PATCH	<code>/api/v1/auth/me/profile-image</code>	Authenticated (any role)	Upload/replace the profile image
POST	<code>/api/v1/auth/refresh-token</code>	Public (valid refresh token required)	Rotate the access + refresh token pair
POST	<code>/api/v1/auth/logout</code>	Public	Clear auth cookies

POST `/api/v1/auth/register`

Access: Public

Starts OTP-based citizen registration. Validates the payload, hashes the password, stores the registration data and a 6-digit OTP in Redis (5 minute TTL), and emails the OTP to the citizen. Rate-limited (5 requests / 15 min per IP).

Request Body:

```
{
  "fullName": "Yesrat Jahan",
  "email": "citizen@example.com",
  "password": "Password@123",
  "phone": "01712345678",
  "address": "Dhaka, Bangladesh"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Check Your Email",
  "data": null
}
```

POST `/api/v1/auth/verify-email`

Access: Public

Verifies the 6-digit OTP against Redis, creates the User + CitizenProfile record, issues access/refresh JWTs (also set as httpOnly cookies), sends a welcome email, and clears the Redis OTP/registration keys.

Request Body:

```
{
  "email": "citizen@example.com",
  "otp": "440353"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Citizen registered successfully",
  "data": {
    "user": {
      "id": "uuid",
      "email": "citizen@example.com",
      "role": "CITIZEN",
      "status": "ACTIVE"
    }
  }
}
```

POST /api/v1/auth/google-login**Access:** Public

Verifies the Google ID token via `google-auth-library`. If the email already exists as a CITIZEN, links the Google account (or logs in); if the email belongs to a STAFF/ADMIN account, the request is rejected. If no account exists, a new CITIZEN account is created with `authProvider: GOOGLE` and `isEmailVerified: true`.

Request Body:

```
{
  "idToken": "<google-id-token>"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Google Login Successfully",
  "data": {
    "accessToken": "<jwt>",
    "refreshToken": "<jwt>"
  }
}
```

POST /api/v1/auth/login**Access:** Public

Authenticates a CITIZEN, STAFF, or ADMIN by email + password (bcrypt compare). Rejects deleted/blocked accounts and Google-only accounts with no password hash. Rate-limited by email (10 requests / 15 min).

Request Body:

```
{
  "email": "citizen@example.com",
  "password": "Password@123"
}
```

Success Response:

```
{
  "success": true,
```

```
"statusCode": 200,
"message": "User logged in successfully",
"data": {
  "accessToken": "<jwt>",
  "refreshToken": "<jwt>"
}
```

POST /api/v1/auth/forgot-password

Access: Public

Sends a 6-digit OTP (5 minute TTL, Redis) to the account email. Rejected for blocked accounts, unverified emails, and Google-only accounts. Rate-limited (5 requests / 15 min).

Request Body:

```
{
  "email": "citizen@example.com"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "A password reset verification code has been sent to your email address.",
  "data": null
}
```

POST /api/v1/auth/reset-password

Access: Public

Verifies the forgot-password OTP, hashes and stores the new password, deletes the Redis OTP key, and sends a confirmation email.

Request Body:

```
{
  "email": "citizen@example.com",
  "otp": "592603",
  "newPassword": "NewPass@123"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Password reset successfully.",
  "data": null
}
```

GET /api/v1/auth/me

Access: Authenticated (any role)

Returns the authenticated user record (password hash stripped) including the role-specific profile (citizen/staff/admin) and, for staff, their department.

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "User profile fetched successfully",
  "data": {
    "id": "uuid",
    "email": "citizen@example.com",
    "role": "CITIZEN",
  }
}
```

```
"citizenProfile": {
  "fullName": "Yesrat Jahan",
  "phone": "01712345678"
}
```

PATCH /api/v1/auth/me

Access: Authenticated (any role)

Updates the role-specific profile fields — `fullName`/`phone`/`address` for a Citizen, `fullName`/`title` for Staff, `fullName` for Admin. At least one field is required.

Request Body:

```
{
  "fullName": "Yesrat Jahan Ety",
  "phone": "017760701872",
  "address": "Chattogram, Bangladesh"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Profile updated successfully",
  "data": {
    "id": "uuid",
    "email": "citizen@example.com"
  }
}
```

PATCH /api/v1/auth/me/profile-image

Access: Authenticated (any role)

Multipart upload (`profileImage` field, image/* only, ≤5MB). Uploads to Cloudinary, deletes the previous image if one existed, and stores the new URL + public ID.

Request Body (multipart/form-data):

```
{
  "profileImage": "<binary file>"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Profile image updated successfully",
  "data": {
    "id": "uuid",
    "profileImage": "https://res.cloudinary.com/.../image.jpg"
  }
}
```

POST /api/v1/auth/refresh-token

Access: Public (valid refresh token required)

Reads the refresh token from the httpOnly cookie (falls back to the request body for non-browser clients), verifies it, and issues a new access token and a rotated refresh token.

Request Body:

```
{
  "refreshToken": "<jwt> (optional if cookie is present)"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "New tokens generated successfully",
  "data": {
    "accessToken": "<jwt>",
    "refreshToken": "<jwt>"
  }
}
```

POST /api/v1/auth/logout**Access:** Public

Clears the `accessToken` and `refreshToken` httpOnly cookies.

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "User logged out successfully",
  "data": null
}
```

2. Departments

Base path: `/api/v1/departments`

City departments (Roads & Infrastructure, Waste Management, Water Supply, Licensing & Permits, etc.) that own categories and service requests. Reads are open to any authenticated user; writes are Admin-only and produce an AuditLog entry.

Method	Endpoint	Access	Description
GET	<code>/api/v1/departments/</code>	Authenticated (any role)	List departments (paginated)
GET	<code>/api/v1/departments/:id</code>	Authenticated (any role)	Get a single department
POST	<code>/api/v1/departments/</code>	Admin	Create a department
PATCH	<code>/api/v1/departments/:id</code>	Admin	Update a department
DELETE	<code>/api/v1/departments/:id</code>	Admin	Soft-delete a department

GET `/api/v1/departments/`

Access: Authenticated (any role)

Paginated list of departments. `includeInactive=true` (Admin only) also returns soft-deleted departments.`

Query / Path Parameters:

- `page` (default 1)
- `limit` (default 10)
- `includeInactive` (Admin only, boolean)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Departments fetched successfully",
  "data": [
    {
      "id": "uuid",
      "name": "Roads & Infrastructure",
      "description": "Potholes, road damage, streetlights"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 4,
    "totalPages": 1
  }
}
```

GET `/api/v1/departments/:id`

Access: Authenticated (any role)

Returns 404 if the department doesn't exist or is soft-deleted.

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Department fetched successfully",
  "data": {
    "id": "uuid",
    "name": "Roads & Infrastructure"
  }
}
```

```
}
```

POST /api/v1/departments/

Access: Admin

Creates a department (unique `name`) and writes a `DEPARTMENT\_CREATED` audit log entry.

Request Body:

```
{
  "name": "Human Resources",
  "description": "Handles hiring, payroll, and employee relations"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Department created successfully",
  "data": {
    "id": "uuid",
    "name": "Human Resources"
  }
}
```

PATCH /api/v1/departments/:id

Access: Admin

Partial update; validates name uniqueness if `name` is changed. Writes a `DEPARTMENT\_UPDATED` audit log entry with before/after values.

Request Body:

```
{
  "name": "Human Resources & People Ops",
  "description": "Updated description"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Department updated successfully",
  "data": {
    "id": "uuid",
    "name": "Human Resources & People Ops"
  }
}
```

DELETE /api/v1/departments/:id

Access: Admin

Sets `deletedAt`. Writes a `DEPARTMENT\_DELETED` audit log entry.

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Department deleted successfully",
  "data": {
    "id": "uuid",
    "deletedAt": "2026-09-07T10:00:00.000Z"
  }
}
```


3. Categories

Base path: `/api/v1/categories`

Request categories belong to a department and define the fee type (FREE/PAID), fee amount, and SLA hours used to compute a request's due date. Reads are open; writes are Admin-only.

Method	Endpoint	Access	Description
GET	<code>/api/v1/categories/</code>	Authenticated (any role)	List categories (paginated, filterable)
GET	<code>/api/v1/categories/:id</code>	Authenticated (any role)	Get a single category
POST	<code>/api/v1/categories/</code>	Admin	Create a category
PATCH	<code>/api/v1/categories/:id</code>	Admin	Update a category
DELETE	<code>/api/v1/categories/:id</code>	Admin	Soft-delete a category

GET `/api/v1/categories/`

Access: Authenticated (any role)

Paginated, filterable by department. Non-admins only see active, non-deleted categories.

Query / Path Parameters:

- `page`
- `limit`
- `departmentId`
- `includeInactive` (Admin only)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Categories fetched successfully",
  "data": [
    {
      "id": "uuid",
      "name": "Trade Licence Renewal",
      "feeType": "PAID",
      "feeAmount": "1500.00",
      "slaHours": 168
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 7,
    "totalPages": 1
  }
}
```

GET `/api/v1/categories/:id`

Access: Authenticated (any role)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Category fetched successfully",
  "data": {
```

```
{
  "id": "uuid",
  "name": "Pothole",
  "feeType": "FREE"
}
```

POST /api/v1/categories/

Access: Admin

`feeAmount` is required and must be > 0 when `feeType` is PAID, and must be omitted when `feeType` is FREE. Writes a `CATEGORY\_CREATED` audit log entry.

Request Body:

```
{
  "departmentId": "uuid",
  "name": "Trade Licence Renewal",
  "feeType": "PAID",
  "feeAmount": 1500,
  "slaHours": 168
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Category created successfully",
  "data": {
    "id": "uuid",
    "name": "Trade Licence Renewal"
  }
}
```

PATCH /api/v1/categories/:id

Access: Admin

Partial update. Fee-type/fee-amount consistency is re-validated against the existing record so a partial update can't leave the category inconsistent. Writes a `CATEGORY\_UPDATED` audit log entry.

Request Body:

```
{
  "name": "Trade Licence Renewal Service",
  "feeType": "PAID",
  "feeAmount": 1750,
  "slaHours": 120,
  "isActive": true
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Category updated successfully",
  "data": {
    "id": "uuid",
    "name": "Trade Licence Renewal Service"
  }
}
```

DELETE /api/v1/categories/:id

Access: Admin

Sets `deletedAt` and `isActive: false`. Writes a `CATEGORY\_DELETED` audit log entry.

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Category deleted successfully",
  "data": {
    "id": "uuid",
    "isActive": false
  }
}
```

4. Service Requests

Base path: `/api/v1/requests`

The core workflow entity. A citizen files a request under a category; PAID categories start in PENDING_PAYMENT (must be paid before entering the queue), FREE categories start directly at SUBMITTED. Staff move an assigned request through IN_PROGRESS → RESOLVED; Admins can reassign or override status at any non-terminal point.

Method	Endpoint	Access	Description
POST	<code>/api/v1/requests/</code>	Citizen	Create a service request (multipart, up to 5 attachments)
GET	<code>/api/v1/requests/search</code>	Authenticated (scoped by role)	Search requests by title / tracking reference
GET	<code>/api/v1/requests/</code>	Authenticated (scoped by role)	List requests (paginated, filterable, sortable)
GET	<code>/api/v1/requests/:id</code>	Authenticated (scoped by role)	Get a single request (full detail)
POST	<code>/api/v1/requests/:id/cancel</code>	Citizen (owner only)	Cancel a request
PATCH	<code>/api/v1/requests/:id/status</code>	Staff (assigned only) / Admin (override)	Transition request status
PATCH	<code>/api/v1/requests/:id/reassign</code>	Admin	Reassign a request to a different staff member / department
POST	<code>/api/v1/requests/:id/reopen</code>	Citizen (owner only)	Reopen a resolved request
POST	<code>/api/v1/requests/:id/attachments</code>	Authenticated (role-scoped)	Add attachment(s) to a request

POST `/api/v1/requests/`

Access: Citizen

Generates a unique `trackingRef` (`NS-<year>-<seq>`). If the category is PAID, the request is created as PENDING_PAYMENT and a payment session is opened automatically (best-effort — the request still exists if session creation fails, and the citizen can retry via `POST /payments/initiate`). Accepts up to 5 image attachments (field name `attachments`).

Request Body (multipart/form-data):

```
{
  "categoryId": "uuid",
  "title": "Pothole on main road",
  "description": "A large pothole has formed near the Agrabad intersection, causing traffic hazards.",
  "address": "Agrabad, Chattogram",
  "latitude": 22.3245,
  "longitude": 91.8116,
  "attachments": "<up to 5 files>"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Service request created successfully",
  "data": {
    "id": "uuid",
    "trackingRef": "NS-2026-000123",
    "status": "SUBMITTED",
    "paymentSession": null
  }
}
```

```
}
```

GET /api/v1/requests/search

Access: Authenticated (scoped by role)

Case-insensitive `contains` match on `title` and `trackingRef`, scoped: Citizens see only their own; Staff see only their department's; Admins see all.

Query / Path Parameters:

- q (required)
- page
- limit

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Search results fetched successfully",
  "data": [
    {
      "id": "uuid",
      "trackingRef": "NS-2026-000123",
      "title": "Pothole on main road"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  }
}
```

GET /api/v1/requests/

Access: Authenticated (scoped by role)

Citizens are scoped to their own requests; Staff to their department; Admins see everything and may additionally filter by `departmentId`.

Query / Path Parameters:

- page, limit
- status (RequestStatus enum)
- departmentId (Admin only)
- categoryId
- overdue=true
- sortBy (createdAt | updatedAt | title | status | slaDueAt)
- sortOrder (asc | desc)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Service requests fetched successfully",
  "data": [
    {
      "id": "uuid",
      "trackingRef": "NS-2026-000123",
      "status": "SUBMITTED",
      "isOverdue": false
    }
  ]
}
```

```
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 42,
    "totalPages": 5
  }
}
```

GET /api/v1/requests/:id

Access: Authenticated (scoped by role)

Includes category, department, attachments, full status history, citizen, and assigned staff. 403 if the requester is not the owning citizen / department staff / admin.

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Service request fetched successfully",
  "data": {
    "id": "uuid",
    "trackingRef": "NS-2026-000123",
    "status": "ASSIGNED",
    "statusHistory": [
      {
        "fromStatus": "SUBMITTED",
        "toStatus": "ASSIGNED",
        "changedBy": "uuid"
      }
    ]
  }
}
```

POST /api/v1/requests/:id/cancel

Access: Citizen (owner only)

Allowed only while status is PENDING_PAYMENT, SUBMITTED, or ASSIGNED. Writes a StatusHistory entry and sets `cancelledAt`. If a COMPLETED payment exists, an automatic refund is attempted — a refund-gateway failure does NOT block the cancellation (it's logged to AuditLog as `PAYMENT\_REFUND\_FAILED` for an Admin to retry manually).

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Service request cancelled successfully",
  "data": {
    "id": "uuid",
    "status": "CANCELLED"
  }
}
```

PATCH /api/v1/requests/:id/status

Access: Staff (assigned only) / Admin (override)

Staff may only move their own assigned request ASSIGNED→IN_PROGRESS or IN_PROGRESS→RESOLVED; a resolution note is required to resolve. Admin may override to any non-terminal status. `slaDueAt` is computed on first ASSIGNED transition. Writes StatusHistory (+ AuditLog for admin overrides) and fires citizen/staff notifications.

Request Body:

```
{
  "toStatus": "IN_PROGRESS",
  "note": "Technician dispatched to the location."
}
```

```
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Request status updated successfully",
  "data": {
    "id": "uuid",
    "status": "IN_PROGRESS"
  }
}
```

PATCH /api/v1/requests/:id/reassign**Access:** Admin

Provide either `staffId` (moves the request into that staff's department, auto-transitions SUBMITTED→ASSIGNED and computes the SLA due date) or `departmentId` alone (unassigns staff, only allowed while not IN_PROGRESS/RESOLVED). Writes StatusHistory (if the status changed) + a `REQUEST\_REASSIGNED` AuditLog entry, and notifies the newly assigned staff member.

Request Body:

```
{
  "staffId": "uuid",
  "reason": "Reassigning to available staff member"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Request reassigned successfully",
  "data": {
    "id": "uuid",
    "assignedStaffId": "uuid",
    "status": "ASSIGNED"
  }
}
```

POST /api/v1/requests/:id/reopen**Access:** Citizen (owner only)

Only valid while status is RESOLVED and within the 3-day window since `resolvedAt`. Transitions back to ASSIGNED.

Request Body:

```
{
  "reason": "The issue has not actually been resolved yet."
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Request reopened successfully",
  "data": {
    "id": "uuid",
    "status": "ASSIGNED"
  }
}
```

POST /api/v1/requests/:id/attachments**Access:** Authenticated (role-scoped)

Citizens may only add EVIDENCE files to their own request; Staff may only add RESOLUTION_PROOF files to a request assigned to them. Capped at 5 attachments total per request (across all uploads).

Request Body (multipart/form-data):

```
{
  "type": "EVIDENCE",
  "attachments": "<up to 5 files>"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Attachment(s) uploaded successfully",
  "data": [
    {
      "id": "uuid",
      "url": "https://res.cloudinary.com/.../evidence.jpg",
      "type": "EVIDENCE"
    }
  ]
}
```

5. Payments

Base path: `/api/v1/payments`

Real payment processing via SSLCommerz and bKash Tokenized Checkout for PAID-category requests. Webhook/redirect/callback endpoints are public (called by the gateway, not the client) and are idempotent — repeat calls never double-complete a payment.

Method	Endpoint	Access	Description
POST	<code>/api/v1/payments/initiate</code>	Citizen (owner only)	Create / recreate a checkout session
GET	<code>/api/v1/payments/:id</code>	Authenticated (scoped; Staff forbidden)	Get a single payment
GET	<code>/api/v1/payments/</code>	Authenticated (scoped; Staff forbidden)	List payments (paginated, filterable, sortable)
PATCH	<code>/api/v1/payments/:id/refund</code>	Admin	Manually trigger / retry a refund
POST	<code>/api/v1/payments/sslcommerz/ipn</code>	Public (SSLCommerz webhook)	SSLCommerz Instant Payment Notification
POST	<code>/api/v1/payments/sslcommerz/success</code>	Public (SSLCommerz redirect)	SSLCommerz success redirect handler
POST	<code>/api/v1/payments/sslcommerz/fail</code>	Public (SSLCommerz redirect)	SSLCommerz failure redirect handler
POST	<code>/api/v1/payments/sslcommerz/cancel</code>	Public (SSLCommerz redirect)	SSLCommerz cancel redirect handler
GET	<code>/api/v1/payments/bkash/callback</code>	Public (bKash callback)	bKash Tokenized Checkout callback

POST `/api/v1/payments/initiate`

Access: Citizen (owner only)

Only valid while the request is PENDING_PAYMENT and has a payable `feeCharged`. Creates a Payment row (PENDING) keyed by a unique `providerRef`, and returns the gateway's checkout URL. Calling again before completion re-creates the session (e.g. to switch provider).

Request Body:

```
{
  "requestId": "uuid",
  "provider": "SSLCOMMERZ"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Payment session created successfully",
  "data": {
    "paymentId": "uuid",
    "checkoutUrl": "https://sandbox.sslcommerz.com/gwprocess/v4/..."
  }
}
```

GET `/api/v1/payments/:id`

Access: Authenticated (scoped; Staff forbidden)

Citizens may only view their own payments. Staff cannot view payments at all (403).

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Payment fetched successfully",
  "data": {
    "id": "uuid",
    "provider": "SSLCOMMERZ",
    "amount": "1500.00",
    "status": "COMPLETED"
  }
}
```

GET /api/v1/payments/

Access: Authenticated (scoped; Staff forbidden)

Citizens are scoped to their own payments; Admins see all.

Query / Path Parameters:

- page, limit
- status
- provider
- sortBy (createdAt | amount | status)
- sortOrder

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Payments fetched successfully",
  "data": [
    {
      "id": "uuid",
      "provider": "SSLCOMMERZ",
      "status": "COMPLETED",
      "amount": "1500.00"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 12,
    "totalPages": 2
  }
}
```

PATCH /api/v1/payments/:id/refund

Access: Admin

Only a COMPLETED payment can be refunded. Dispatches to the correct gateway (SSLCommerz / bKash) based on `provider`. Failures are recorded (`PAYMENT\_REFUND\_FAILED`) and re-thrown so the Admin sees the exact reason; success writes `PAYMENT\_REFUNDED` (+ an optional `PAYMENT\_MANUAL\_REFUND\_REASON` entry).

Request Body:

```
{
  "reason": "Duplicate charge – refunding on citizen request"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Payment refunded successfully",
}
```

```
{
  "data": {
    "id": "uuid",
    "status": "REFUNDED"
  }
}
```

POST /api/v1/payments/sslcommerz/ipn

Access: Public (SSLCommerz webhook)

Server-to-server webhook. Re-validates the transaction with SSLCommerz's `validate` API (status + amount + currency) before marking the Payment COMPLETED and the ServiceRequest SUBMITTED, in the same DB transaction.

Request Body (application/x-www-form-urlencoded):

```
{
  "tran_id": "NS-xxxxxxxx-1234567890",
  "val_id": "sample-val-id",
  "status": "VALID"
}
```

Success Response:

```
{
  "received": true
}
```

POST /api/v1/payments/sslcommerz/success

Access: Public (SSLCommerz redirect)

Browser redirect target after a successful checkout. Independently re-validates and completes the payment (idempotent alongside the IPN), then 302-redirects to the frontend success page.

Request Body (application/x-www-form-urlencoded):

```
{
  "tran_id": "NS-xxxxxxxx-1234567890",
  "val_id": "sample-val-id"
}
```

POST /api/v1/payments/sslcommerz/fail

Access: Public (SSLCommerz redirect)

Marks a PENDING payment as FAILED (idempotent) and redirects to the frontend failure page.

Request Body (application/x-www-form-urlencoded):

```
{
  "tran_id": "NS-xxxxxxxx-1234567890"
}
```

POST /api/v1/payments/sslcommerz/cancel

Access: Public (SSLCommerz redirect)

Marks a PENDING payment as FAILED and redirects to the frontend cancel page.

Request Body (application/x-www-form-urlencoded):

```
{
  "tran_id": "NS-xxxxxxxx-1234567890"
}
```

GET /api/v1/payments/bkash/callback

Access: Public (bKash callback)

Single callback URL; success/failure/cancel are differentiated by the `status` query param. On a non-cancel/failure callback, calls `executePayment`, cross-checks `transactionStatus === 'Completed'` and the amount before completing, then redirects to the frontend.

Query / Path Parameters:

- paymentID (required)
- status (success | failure | cancel)

6. Feedback

Base path: **/api/v1/feedbacks**

One rating (1–5) + optional comment per service request, only allowed once the request reaches RESOLVED or CLOSED, and enforced 1:1 by a unique constraint on `requestId`.

Method	Endpoint	Access	Description
POST	/api/v1/feedbacks/	Citizen (owner only)	Submit feedback for a resolved/closed request
GET	/api/v1/feedbacks/	Authenticated (scoped by role)	List feedback (paginated, filterable)
GET	/api/v1/feedbacks/:requestId	Authenticated (scoped by role)	Get feedback for a specific request

POST /api/v1/feedbacks/

Access: Citizen (owner only)

Rejects if the request isn't the citizen's own, isn't RESOLVED/CLOSED, or already has feedback.

Request Body:

```
{
  "requestId": "uuid",
  "rating": 5,
  "comment": "Resolved quickly and professionally."
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Feedback submitted successfully",
  "data": {
    "id": "uuid",
    "rating": 5
  }
}
```

GET /api/v1/feedbacks/

Access: Authenticated (scoped by role)

Citizens see only their own; Staff see their department's; Admins see all (optionally filtered by department).

Query / Path Parameters:

- page, limit
- rating (1-5)
- departmentId (Admin only)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Feedback list fetched successfully",
  "data": [
    {
      "id": "uuid",
      "rating": 5,
      "comment": "Great service"
    }
  ],
  "meta": {
```

```
{
  "page": 1,
  "limit": 10,
  "total": 8,
  "totalPages": 1
}
```

GET /api/v1/feedbacks/:requestId

Access: Authenticated (scoped by role)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Feedback fetched successfully",
  "data": {
    "id": "uuid",
    "rating": 5,
    "comment": "Great service"
  }
}
```

7. Admin

Base path: `/api/v1/admin`

Platform-operator endpoints: provisioning Staff/Admin accounts, blocking/unblocking users, changing roles, auditing, and dashboard analytics. Every mutating action here writes to AuditLog.

Method	Endpoint	Access	Description
POST	<code>/api/v1/admin/staff</code>	Admin	Provision a Staff or Admin account
GET	<code>/api/v1/admin/users</code>	Admin	List / search all users (paginated, filterable, sortable)
PATCH	<code>/api/v1/admin/users/:id/status</code>	Admin	Block or unblock a user
PATCH	<code>/api/v1/admin/users/:id/role</code>	Admin	Change a user's role (STAFF ↔ ADMIN)
GET	<code>/api/v1/admin/audit-logs</code>	Admin	List audit logs (paginated, filterable)
GET	<code>/api/v1/admin/dashboard-stats</code>	Admin	Aggregate platform statistics

POST `/api/v1/admin/staff`

Access: Admin

Creates a User with a generated temporary password (``mustChangePassword: true``, ``isEmailVerified: true`` — provisioned accounts skip OTP), and the matching StaffProfile/AdminProfile. ``departmentId`` is required when ``role`` is STAFF. Emails the credentials to the provided ``personalEmail`` (best-effort — failure doesn't roll back account creation).

Request Body:

```
{
  "fullName": "Rahim Uddin",
  "personalEmail": "rahim.personal@gmail.com",
  "organizationEmail": "staff.roads2@nagar-sheba.com",
  "role": "STAFF",
  "departmentId": "uuid",
  "title": "Field Technician"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Account provisioned successfully",
  "data": {
    "userId": "uuid",
    "organizationEmail": "staff.roads2@nagar-sheba.com"
  }
}
```

GET `/api/v1/admin/users`

Access: Admin

Query / Path Parameters:

- page, limit
- role (CITIZEN|STAFF|ADMIN)
- status (ACTIVE|BLOCKED)
- search (email contains)
- sortBy (createdAt|email|role|status)

- sortOrder

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Users fetched successfully",
  "data": [
    {
      "id": "uuid",
      "email": "citizen@example.com",
      "role": "CITIZEN",
      "status": "ACTIVE"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 120,
    "totalPages": 12
  }
}
```

PATCH /api/v1/admin/users/:id/status

Access: Admin

Admin accounts cannot be blocked/unblocked via this endpoint, and an Admin cannot change their own status. Writes `USER\_BLOCKED` / `USER\_UNBLOCKED` to AuditLog.

Request Body:

```
{
  "status": "BLOCKED"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "User status updated successfully",
  "data": {
    "id": "uuid",
    "status": "BLOCKED"
  }
}
```

PATCH /api/v1/admin/users/:id/role

Access: Admin

Citizen accounts cannot be promoted through this endpoint, an Admin cannot change their own role, and the target must not already hold the requested role. Drops the profile tied to the old role (unassigning any requests still assigned to an outgoing Staff member) and creates the profile for the new role, all inside one transaction. Writes `USER\_ROLE\_CHANGED` to AuditLog.

Request Body:

```
{
  "role": "STAFF",
  "departmentId": "uuid",
  "title": "Senior Field Technician"
}
```

Success Response:

```
{
  "success": true,
  "statusCode": 200,

```

```
{
  "message": "User role updated successfully",
  "data": {
    "id": "uuid",
    "role": "STAFF"
  }
}
```

GET /api/v1/admin/audit-logs

Access: Admin

Query / Path Parameters:

- page, limit (default 20)
- entityType
- actorId

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Audit logs fetched successfully",
  "data": [
    {
      "id": "uuid",
      "action": "REQUEST_STATUS_OVERRIDDEN",
      "entityType": "ServiceRequest",
      "actorId": "uuid"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 340,
    "totalPages": 17
  }
}
```

GET /api/v1/admin/dashboard-stats

Access: Admin

Users by role, requests by status + overdue count, total completed payment revenue + pending count, and average feedback rating — computed via parallel Prisma aggregations.

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Dashboard stats fetched successfully",
  "data": {
    "users": {
      "total": 120,
      "citizens": 100,
      "staff": 15
    },
    "requests": {
      "total": 340,
      "overdue": 12,
      "byStatus": {
        "SUBMITTED": 20,
        "ASSIGNED": 15,
        "RESOLVED": 200
      }
    }
  },
}
```

```
    "payments": {  
      "totalRevenue": "185000.00",  
      "pending": 3  
    },  
    "feedback": {  
      "averageRating": 4.6  
    }  
  }  
}
```

8. Notifications

Base path: **/api/v1/notifications**

In-app notification log (status changes, assignments, payment events, feedback). Notification creation is best-effort/fire-and-forget from other modules and never blocks or rolls back the triggering business transaction.

Method	Endpoint	Access	Description
GET	/api/v1/notifications/me	Authenticated (any role)	List my notifications (paginated)
GET	/api/v1/notifications/unread-count	Authenticated (any role)	Get unread notification count
PATCH	/api/v1/notifications/:id/read	Authenticated (owner only)	Mark one notification as read
PATCH	/api/v1/notifications/read-all	Authenticated (any role)	Mark all my notifications as read

GET /api/v1/notifications/me

Access: Authenticated (any role)

Query / Path Parameters:

- page, limit
- isRead (true|false)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Notifications fetched successfully",
  "data": {
    "notifications": [
      {
        "id": "uuid",
        "type": "REQUEST_STATUS_CHANGED",
        "isRead": false
      }
    ],
    "unreadCount": 3
  },
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 3,
    "totalPages": 1
  }
}
```

GET /api/v1/notifications/unread-count

Access: Authenticated (any role)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Unread notification count fetched successfully",
  "data": {
    "unreadCount": 3
  }
}
```

```
}
```

PATCH /api/v1/notifications/:id/read

Access: Authenticated (owner only)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Notification marked as read",
  "data": {
    "id": "uuid",
    "isRead": true
  }
}
```

PATCH /api/v1/notifications/read-all

Access: Authenticated (any role)

Success Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "All notifications marked as read",
  "data": {
    "updatedCount": 3
  }
}
```

6. Data Model Summary

PostgreSQL via Prisma, modeled with a multi-file schema. Key entities:

- User — single table for all 3 roles, linked 1:1 to exactly one profile (CitizenProfile / StaffProfile / AdminProfile) based on role.
- Department → Category (fee type, SLA hours) → ServiceRequest.
- ServiceRequest — the central entity: tracking reference, geolocation, status, SLA due date, overdue flag, soft delete.
- StatusHistory — full audit trail of every status transition (fromStatus → toStatus, actor, note).
- Payment — 1:1 with a ServiceRequest; tracks provider, provider reference, amount, status, paid/refunded timestamps.
- Attachment — evidence (citizen) or resolution proof (staff), stored on Cloudinary.
- Feedback — 1:1 rating + comment, only allowed after RESOLVED/CLOSED.
- AuditLog — actor, action, entity, before/after snapshot — written for all admin-level overrides.
- Notification — in-app/email notification log.

All list-heavy tables carry indexes on their most-queried columns (status, citizenId, departmentId, assignedStaffId, categoryId, isOverdue, createdAt), and soft-deletes use a nullable deletedAt timestamp rather than hard deletes.

Enum Reference

Enum	Values
Role	CITIZEN, STAFF, ADMIN
AccountStatus	ACTIVE, BLOCKED
AuthProvider	CREDENTIAL, GOOGLE
FeeType	FREE, PAID
RequestStatus	PENDING_PAYMENT, SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED, CANCELLED
PaymentProvider	SSLCOMMERZ, BKASH
PaymentStatus	PENDING, COMPLETED, FAILED, CANCELLED, REFUNDED
AttachmentType	EVIDENCE, RESOLUTION_PROOF
NotificationChannel	EMAIL

7. Error Codes Reference

Status	Name	Typical Cause
400	Bad Request	Validation failed, malformed payload, or a business rule was violated
401	Unauthorized	Missing/invalid/expired token, or invalid login credentials
403	Forbidden	Authenticated, but the role/ownership doesn't permit this action
404	Not Found	The resource (or route) doesn't exist, or is soft-deleted
409	Conflict	The action conflicts with the resource's current state (e.g. an illegal status transition)
502	Bad Gateway	The upstream payment gateway (SSLCommerz/bKash) returned a failure
500	Internal Server Error	Unhandled/unexpected error

— End of Document —